

Omni:

A Tool For Transcribing Programming Languages



Justin Grubbs
Hendrix College

A capstone project submitted for the degree of
Bachelor of Computer Science

Hendrix College 2024

Abstract

The most popular programming languages taught in universities are Python, Java, and C++. Although these languages are all imperative, object-oriented languages, they have many practical differences and are often used in distinct programming domains. I created a program using the functional programming language Haskell that would allow a user to transcribe one of these programming languages to another. This tool, optimistically named “Omni,” could help students better understand how these programming languages differ beyond syntax.

Contents

1	Introduction	1
1.1	Implemented Languages	2
2	Background	3
2.1	Terminology	3
2.1.1	Compile vs Transpile	3
2.1.2	Statements and Expressions	4
2.2	Implemented Features	4
3	Abstract Syntax Trees	5
3.1	Omni’s Abstract Syntax Tree	6
3.1.1	Direct Transcriptions	8
4	Parsing	9
4.1	Parsing in Omni	9
4.2	Problems With Importing Parsers	9
4.2.1	Negatives of Importing Parsers	10
4.2.2	Benefits of Importing Parsers	10
5	Type Systems	12
5.1	Type Checking: Static vs Dynamic	12
5.1.1	How Expressions Are Typed: Explicit vs Implicit	13
5.1.2	Type Safety	13
5.2	Relevance to Omni	13
5.2.1	Python’s Implicit Typing	14

5.2.2	Invalid Statements	14
5.2.3	Mutability	15
5.2.4	Issues with Functions	15
5.3	Type Checking in Omni	16
5.3.1	Type Inference	17
5.3.1.1	Inference on Functions	18
5.3.1.2	Unification-Based Type Inference	18
5.3.2	Elaboration	19
5.3.2.1	Resolving Type Mutations	19
5.3.2.2	Issues From Allowing Type Mutations	20
6	Conclusions	21
6.1	Features Excluded From Omni	21
6.2	Potential Improvements to Omni	21
6.3	Future Languages to Implement	22
	Bibliography	23

List of Figures

1.1	Omni Example	1
3.1	Natural Language AST	5
3.2	Basic Arith AST	6
3.3	Omni’s AST Reduced	7
4.1	<i>Other</i> Data Constructors in Omni - Extends Figure 3.3	11
5.1	Implicit/Explicit Dichotomy	14
5.2	Abstract Syntax of Figure 5.1	14
5.3	Type Mutations - Invalid	15
5.4	Type Mutations - Valid	15
5.5	Method Overloading	16
5.6	Example Transpilation with Method Overloading	16
5.7	Valid Python Conditional	17
5.8	Inference on Base Types	17
5.9	Consequence of Statement Elaboration	19
5.10	Type Mutations	19
5.11	Problems With Type Mutability	20

Chapter 1

Introduction

Learning a programming language can be a daunting task. The differences in syntax alone can be enough to dissuade a novice programmer. I created Omni to alleviate some of these issues. Omni works as follows: the user specifies the desired language translation and inputs the initial file. Omni will then parse the given code, transform it into abstract syntax, and, in some cases, perform some type checking. Finally, Omni takes the abstracted code, prints it to a file in the desired syntax, and outputs the file. The entire source code for Omni can be found on [GitHub](#).

<pre>def main(): x = add(3,2) y = mul(x,10) z = True if z: x += 1 x = "Squirrel" def add(x,y): return x + y def mul(x,y): return x * y main()</pre>	<pre>public static void main(String[] args) { int x = add(3,2); int y = mul(x,10); boolean z = true; if (z) { x += 1; } String A = "Squirrel"; } public static int add(int x, int y) { return x + y; } public static int mul(int x, int y) { return x * y; }</pre>
---	--

Figure 1.1: Omni Example

This paper will follow the developmental process of Omni. In doing so, I will showcase some of the concepts that differentiate various popular programming languages from one

another, and I hope to provide some insight into the basic construction of a programming language.

1.1 Implemented Languages

As its name implies, Omni was designed with the intent to be extendable to support any imperative¹ programming language. This is not currently the case. There are many cases that will require special consideration and work to properly transpile. For example, because C++ lacks a garbage collector, transpiling from Python to C++ would require special work to deal with memory allocation and deallocation. Currently, Omni currently only supports Python and Java. Aside from my familiarity with them and their general popularity in software development, these languages were chosen because of the differences between their type systems, which will be elaborated on later.

¹Although attempting to implement a functional language would be interesting, it is unfortunately vastly out of scope for this project.

Chapter 2

Background

Omni is intended to be used by people who seek to learn **another** programming language. Thus, this paper assumes that readers are proficient in at least one programming language. Because Omni is written in Haskell, there will unavoidably be snippets of Haskell code throughout this paper; however, experience in Haskell is **not** assumed. In this paper, Haskell code will always be accompanied by a natural language explanation.

2.1 Terminology

There are a few terms used throughout this paper that must be defined. Typically, this is done whenever the terms are introduced, but there are a few terms used throughout this paper that should be defined here.

2.1.1 Compile vs Transpile

Compilation is a term used to describe the process of transforming a programming language into another language; however, it is almost solely used to describe the process of transforming a high-level language into machine code. Thus, throughout this paper, the term “transpile” is often used to describe the general process of Omni: transforming a high-level language into another high-level language.

2.1.2 Statements and Expressions

Programming language constructs are often¹ divided into two categories: statements and expressions. Intuitively, statements can be understood as instructions that affect the “state” of a program, such as assignments and declarations. Conversely, expressions are solely evaluated to a single value, such as binary operations. These terms are used fairly interchangeably throughout this paper, but it is important to note the overall distinction.

2.2 Implemented Features

Due to the nature of this project and the limited time allotted, Omni encompasses only a small subset of Python and Java features, respectively. A non-exhaustive list of supported features is shown in the following table:

Supported Features of Omni	
Expressions	Statements
Numerals	Assignments
Booleans	Declarations
Strings	Blocks
Characters	Conditionals
Variables	Loops
Arrays	Functions
Various operators	
...	...

¹Functional programming languages, for example, often do not have any concept of statements, and instead involve chaining expressions together.

Chapter 3

Abstract Syntax Trees

Omni is designed to be an intermediate language, that is to serve as a sort of bridge between languages. At its core, it is represented by an abstract syntax tree (AST). ASTs represent the logical structure of the programming language and discard any non-essential syntactical clutter. Consider the following AST to represent a natural language with nouns, verbs, and periods to denote the end of the statement.¹

```
data PartOfSpeech where
  Noun :: String -> PartOfSpeech
  Verb :: String -> PartOfSpeech
```

Figure 3.1: Natural Language AST

Looking past the Haskell syntax, a new type, `PartOfSpeech`, is defined to have two constructors: `Noun` and `Verb`, both of which take a string as an argument. Thus, the sentence “**Bill hikes.**” would be represented as the following:

Noun (“Bill”) Verb (“hikes”)

Likewise, consider the following AST to represent a language with only integers and addition.

¹In this example, a period just denotes the end of a sentence. In reality, punctuation such as a question mark would affect the semantics in question, but for the sake of this example, they will not be considered.

```
data Arith where
  Num :: Int -> Arith
  Add :: Arith -> Arith -> Arith
```

Figure 3.2: Basic Arith AST

The expression `3 + 2`; would be represented as the following:

Add (Num (3)) (Num (2))

Notably, both the period in the natural language example and the semi-colon in the Arith example were discarded, as neither affect the semantics.

3.1 Omni's Abstract Syntax Tree

I chose to structure Omni as an intermediate language.² That is, I defined an abstract syntax tree that is general enough to support Python, Java, and (ideally) most other imperative languages. This has clear benefits when implementing new languages to Omni, as one would only need to explicitly define the connection between the new language and the intermediate language. However, this is not without downsides. It needs to be explicitly defined how this intermediate language is transformed into each implemented language's respective syntax. Furthermore, in making this intermediate language generalizable enough to support the many distinct languages it is intended to support, some information is lost in translation. This loss of information could be mitigated with a sufficiently complex abstract syntax tree, but designing such a tree would have been out of scope for this project. A **considerably reduced** version of Omni's AST is shown below.

²There are existing examples of this in which an intermediate language is used to optimize compiling languages to machine code. The most notable is the GNU Compiler Collection (GCC).[1]

```

data Stmt where
  Assign    :: [Ident] -> Expr -> Stmt
  Declare   :: Type -> [Ident] -> Maybe Expr -> Stmt
  Block     :: Prog -> Stmt
  IfThen    :: Expr -> Stmt -> Stmt
  IfElse    :: Expr -> Stmt -> Stmt -> Stmt
  While     :: Expr -> Stmt -> Stmt
  FunDecl   :: Ident -> [Args] -> Type -> Stmt -> Stmt
  ExprStmt  :: Expr -> Stmt
  Return    :: Maybe Expr -> Stmt
  ...
deriving (Show,Eq)

data Expr where
  Lit      :: Literal -> Expr
  Var      :: Ident -> Expr
  Array    :: [Expr] -> Expr
  Bin      :: BOp -> Expr -> Expr -> Expr
  Call     :: [Ident] -> [Expr] -> Expr
  ...
deriving (Show,Eq)

data Literal where
  Int      :: Integer -> Literal
  Bool     :: Bool -> Literal
  Char     :: Char -> Literal
  Str      :: String -> Literal
  ...
deriving (Show, Eq)

```

Figure 3.3: Omni’s AST Reduced

3.1.1 Direct Transcriptions

Transcribing directly from language to language would require explicitly defining how data constructors in Python would be translated to Java and how they would be translated from Java to Python. Viewed optimistically, this does not seem like an awful idea, as it could reduce how much information is lost in the process of abstraction. However, that optimism quickly turns into horror when considering that for every newly implemented language, connections would need to be explicitly defined for every previously implemented language. That being said, I think this is worth noting as implementing this in specific cases where the languages are **sufficiently similar** could result in a superior output³ than Omni's generalized AST.

³The example that most readily comes to mind would be Java and C#, but I am not experienced enough with C# to confidently make the assertion that this would result in a superior output.

Chapter 4

Parsing

Parsing is the actual process of reading source code, analyzing it, stripping it of unnecessary syntax (such as semi-colons in Java), and encoding it into an abstract syntax tree. In essence, parsing is the act of separating semantics from syntax.

4.1 Parsing in Omni

I started by creating my own parsers for Python and Java. However, this was a longer process than expected and far more tedious due to Python's syntax. Parsing newlines and leading space in Python is a non-trivial task, and was consuming more time and effort than it was worth. Furthermore, it was clear that Omni was not going to be able to encode the entirety of these languages in the time period allotted. This means that there would be valid pieces of Python and Java code (such as classes and objects) that would be rejected with a parsing error. Thus, I decided to scrap my custom parsers and import them instead. The packages that are used are *language-java*[2] by Niklas Broberg and *language-python*[3] by Bernard James Pope.

4.2 Problems With Importing Parsers

Although this change was very beneficial and allowed more time and effort to be spent on the essence of the project itself, it did not come without issues. The imported parsers each came with a distinct abstract syntax tree, both of which were far more complex than my own. Thus, I needed to convert each imported AST to my own custom AST, and, in the process, prune the non-essential pieces.

The following is the result of parsing `x = 3` in Python and Java, respectively.

```
Assign {assign_to = [Var {var_ident = Ident {ident_string = "x",
ident_annot = SpanPoint {span_filename = "figure_4_1.py", span_row = 1,
span_column = 1}}, expr_annot = SpanPoint {span_filename =
"figure_4_1.py", span_row = 1, span_column = 1}}], assign_expr = Int
{int_value = 3, expr_literal = "3", expr_annot = SpanPoint
{span_filename = "figure_4_1.py", span_row = 1, span_column = 5}},
stmt_annot = SpanCoLinear {span_filename = "figure_4_1.py", span_row =
1, span_start_column = 1, span_end_column = 5}}
```

and

```
VarDecl (VarId (Ident "x")) (Just (InitExp (Lit (Int 3))))
```

both respectively become

```
Assign "x" (Lit (Int 3))
```

4.2.1 Negatives of Importing Parsers

Furthermore, in importing parsers, we are limited only to the information explicitly gathered by the imported parser. Since Omni’s AST is necessarily more general than a typical AST, it is not obvious how this is problematic. However, suppose we wanted to enrich our AST to include specifically where in the original file each construct came from in order to produce stronger, more specific error messages.¹ As showcased above, the imported Python parser collects this information in the form of “source span;” however, the imported Java parser does not. Thus, Omni is, in general, limited only to information that is gathered by **each** and **every** imported parser.

4.2.2 Benefits of Importing Parsers

Most obviously, importing parsers makes implementing new features much easier, as we no longer have to worry about all of the very subtle quirks that each language has. More importantly, we are now able to catch all constructs that are not explicitly implemented in Omni. This is done by adding an *Other* data constructor to each algebraic data type in Omni’s AST, shown below.

¹This is certainly a reasonable inclusion, as Omni’s error messages are fear-inducing.

```
data Stmt where
  ...
  OtherS    :: String -> Stmt
  ...
  deriving (Show,Eq)

data Expr where
  ...
  OtherE    :: String -> Expr
  ...
  deriving (Show,Eq)

data Literal where
  ...
  OtherL    :: String -> Literal
  ...
  deriving (Show, Eq)
```

Figure 4.1: *Other* Data Constructors in Omni - Extends Figure 3.3

With this addition to our AST, we can now throw a “conversion” error. That is, we print the unimplemented construct and explain that it is not supported by Omni.

Chapter 5

Type Systems

After parsing the source code and encoding it into the abstract syntax of Omni, we are not necessarily able to immediately turn it into the concrete syntax of the desired language. As alluded to earlier, Python and Java were specifically selected because of their differing type systems and ways that their expressions are typed. We are all familiar with types and type systems, but it is worth taking a moment to consider actually why they exist. Types are a construct to help us distinguish between “good” and “bad” programs. What specifically distinguishes good programs from bad ones is a per-language choice, but **generally**, good programs can be thought of as ones that do not crash, do not access memory they are not supposed to, and do not allow nonsensical operations, such as `12 + True` [4]. This way of defining bad programs is certainly **not**¹ universal, and is typically split into two systems of type checking: static and dynamic.

5.1 Type Checking: Static vs Dynamic

A dynamically checked² language, such as Python, is one in which typechecking is done during runtime. Conversely, a statically checked language, such as Java, is a language in which typechecking is done during compilation. This means that in a static language, the types of the entire program are known before runtime. This is typically accomplished either through type inference or by requiring explicit type annotations from the programmer.

¹`12 + True`, for instance, is a valid operation in Python and will not result in an error.

²It is standard for this to be referred to as ‘dynamically typed’ and ‘statically typed,’ but these terms are “arguably misnomers.” [5, p. 2-3]

5.1.1 How Expressions Are Typed: Explicit vs Implicit

In general, explicitly typed languages use type annotations to help guide the typechecker. Implicitly typed languages are ones that require the typechecker to infer or reconstruct the types of expressions.[5, p. 101] Contrary to my understanding before undertaking this project, static languages are **typically** but not **necessarily** explicitly typed. Haskell, for instance, is a statically checked language with implicitly typed expressions.³ Most dynamic languages are implicitly typed; however, many support optional explicit typing.

5.1.2 Type Safety

It is worth noting that, among other things, type systems are also distinguished by the extent in which they prevent run-time type errors. In other words, languages are “safe” if the type system limits the amount of run-time type errors that can occur. Naturally, a language whose type system does not limit run-time type errors is “unsafe.”⁴ Like the other aspects of type systems, there are varying degrees of type safety, but generally languages are considered safe if they are memory safe and prevent illegal operations, such as adding a character and an integer. Thus, both Java and Python are safe languages, and languages like C are unsafe due to unsafe actions with memory allocation and deallocation. Because Java and Python are both safe languages, this aspect is not yet relevant in Omni, but would certainly be a concern moving forward.

5.2 Relevance to Omni

Most of the time working on Omni was spent on managing the relationships between these various aspects of type systems. Since Java is a static language, the types are known at compile time. Thus, when transcribing from Java source code, we assume that there were no compilation errors and do not typecheck the source code ourselves. It would not take a considerable amount of time or effort to do some sanity typechecking for Java source code, but, for the current set of features that Omni supports, the benefit would be minimal. Thus, the remainder of this chapter will solely be considering transcriptions from Python source code, as this case is far more interesting. The remainder of this section will showcase

³Like many other implicitly typed languages, Haskell supports optional explicit typing.

⁴It is common for this to be referred to as being “strongly” or “weakly” typed, but this terminology is slightly ambiguous.

basic Python examples and their Java equivalents, in the hope of building an intuition into some of the **many** problems these various aspects of type systems cause for Omni before discussing how to actually resolve these issues.

5.2.1 Python’s Implicit Typing

By far the most obvious issue is Python’s implicit typing. Consider the following:

<pre>x = 12 y = True z = "Squirrel"</pre>	<pre>int x = 12; boolean y = true; String z = "Squirrel";</pre>
---	---

Figure 5.1: Implicit/Explicit Dichotomy

Before we can output valid Java source code, we must first reconstruct the types of all the expressions in the original source code. In other words, we must infer all of the types from the context of the source program.

5.2.2 Invalid Statements

There is another, slightly less obvious issue that arises from Figure 5.1. Consider the following abstract syntax representation of Figure 5.1:⁵

<pre>Assign ["x"] (Int 12), Assign ["y"] (Bool True), Assign ["z"] (Str "Squirrel")</pre>	<pre>Declare TyInt ["x"] (Int 12), Declare TyBool ["y"] (Bool True), Declare TyStr ["z"] (Str "Squirrel")</pre>
---	---

Figure 5.2: Abstract Syntax of Figure 5.1

Because Java requires explicit type annotations, not only must we reconstruct the types of all the expressions, we must also modify the abstract syntax to fit the concrete syntax of the target language. I refer to this process as elaboration, as we are adding additional information that was left implicit in the Python source code. This extends well beyond just

⁵In order to have enough space to display abstract syntax, parts that aren’t relevant to the matter at hand may be elided, such as the literal wrappers and Maybe datatype.

assignments and declarations, as there are many instances in which elaborating Python source code is required.

5.2.3 Mutability

Both Python and Java allow for the values of variables to be mutated by computations. To my dismay, Python one-ups Java by not only allowing the values to be mutated, but by also allowing the **types** of variables to be mutated. Consider the following adaptation to Figure 5.1:

<pre>x = 12 x = True x = "Squirrel"</pre>	<pre>int x = 12; boolean x = true; String x = "Squirrel";</pre>
---	---

Figure 5.3: Type Mutations - Invalid

Disregarding the fact that x is declared multiple times and would result in an error even if the types were unified, the above Java code will result in a type error, since x is already of type *int*. Now, consider the following modification to Figure 5.3:

<pre>x = 12 x = True x = "Squirrel"</pre>	<pre>int x = 12; boolean A = true; String B = "Squirrel";</pre>
---	---

Figure 5.4: Type Mutations - Valid

Because setting x to be an int, boolean, and string in Java would result in type errors, we must “refactor” variables whenever their types are mutated. This is not a problem for simple programs like the one showed above, but this has fairly drastic consequences when slightly more sophisticated programs are considered. These issues can range from minor semantical differences when conditionals are considered to infinite loops when while loops are considered.

5.2.4 Issues with Functions

There are many problems that arise when implementing functions into Omni. Consider the following example:

<pre>def add(x,y): return x+y add(3,2) add("Hello", " world!")</pre>	<pre>public static int add(int x, int y) { return (x+y); } add(3,2); add("Hello", " world!");</pre>
--	---

Figure 5.5: Method Overloading

Currently, attempting to transpile the Python source code with Omni would result in a type error, as the type of *add* and its arguments are inferred to be *int*, and thus a type mismatch is thrown when *add* is called with arguments of type *string*. The solution is actually fairly simple. Java, alongside many other object-oriented languages, support a feature called “method overloading.” In essence, this allows multiple function/method declarations with the same identifier, so long as the types of the arguments differ in some way from previous declarations. Thus, when a function is called with differently typed arguments from a previous call, Omni should eschew from throwing a type error. Instead, it should attempt to continue the inference process with the new argument types. If the end result of inferring the function body with the new argument types is well-typed, the function is valid and should be allowed. Otherwise, Omni should throw a type error as before. Naturally, this would slightly complicate the process of inferring function calls, but the benefit would certainly outweigh the cost.

```
public static int add(int x, int y) {
    return (x+y); }
public static String add(String x, String y) {
    return (x+y); }
add(3,2);
add("Hello", " world!");
```

Figure 5.6: Example Transpilation with Method Overloading

5.3 Type Checking in Omni

As mentioned before, Omni resolves these issues with a process called elaboration. We take the progression of statements that was parsed from the source code and iterate through it, inferring the types of expressions and expanding/modifying statements when necessary. At

the core of a type inference system is a set of typing rules that dictate how statements and expressions are typed. In Omni, for instance, a conditional is well-typed if and only if the condition is of type *Bool* and if the types of the two branches are equivalent.⁶

Because we are transpiling Python source code to Java, we need to infer the types of Python expressions as if it were a statically checked language. This necessarily requires limiting many Python computations that would otherwise be valid. Consider the following:

```
if True:
    return 12
else:
    return "Squirrel"
```

Figure 5.7: Valid Python Conditional

Because Python is dynamically checked, the example in Figure 5.5 would be valid Python source code. However, because the types of the two branches are not equivalent, it is not consistent with Omni’s typing rule for conditionals and thus must be rejected with a conversion error.

5.3.1 Type Inference

Type inference is the process of inferring or reconstructing type annotations from the context of the source program. This process is possible because we are able to infer the types of literals, such as integers, characters, booleans, etc., without requiring additional information. These are commonly referred to as base types.⁷

```
inferLit :: Literal -> Type
inferLit (Int i)    = TyInt
inferLit (Bool b)   = TyBool
inferLit (Str s)    = TyStr
...
```

Figure 5.8: Inference on Base Types

⁶There is a concise mathematical notation to describe these typing rules. However, for unfamiliar readers, the price of conciseness is comprehensibility. Thus, it will not be used throughout this paper.

⁷This is commonly also referred to as “atomic” or “primitive” types.

Because we can infer base types, we can infer the types of complex expressions by recursively inferring the type of each sub-expression. Once its sub-expressions have been reduced to their base types, we can then check if the original expression is well-typed according to our typing rules and, if so, type it accordingly.

5.3.1.1 Inference on Functions

The type inference system that is described in this section encounters issues when functions are considered. When transpiling functions from Python to Java, we must infer the types of its arguments, its return type, and all the types in the function body. For functions without arguments, we can perform inference with no issues. However, for functions with arguments, the types of the arguments must be known before we can correctly perform inference on the function body. Thus, before we can perform inference on a function with arguments, we must first infer the types of the expressions used when the function is called. In most cases, this resolves our issues. Unfortunately, however, if a function with arguments is defined but never called, Omni is unable to proceed. Thus, Omni could⁸ require that a function with arguments is either called somewhere in the source code or that its arguments contain explicit type annotations. With this change, we could now successfully perform inference on functions. Occasionally requiring explicit type annotations for function arguments is an unfortunate limitation of Omni, but one that could be resolved with a more robust inference system.

5.3.1.2 Unification-Based Type Inference

It is important to note that the aforementioned issues with inferring function types are not a weakness of type inference as a whole, but solely a weakness of Omni’s rudimentary type inference system. A better implementation of type inference would be to use a unification-based algorithm. In essence, this algorithm would entail generating sets of constraints for expressions that cannot be outright inferred, then incrementally resolving those constraints in order to find the **most general** type. It is important to note that, in languages that utilize type unification, the “most general” type is often polymorphic. In Omni, special work would need to be done to *monomorphize* types in languages which don’t support

⁸Because Omni is expected to be extended with a more robust inference system that would nullify the need for this modification, this suggestion is not actually reflected in the current version of Omni. Instead, an error is thrown.

polymorphism. Notably, unification-based type inference is implemented in the type systems of many functional programming languages, including Haskell.[6]

5.3.2 Elaboration

Resolving the issue of invalid statements from 5.2.2 is actually fairly trivial, but can cause some very subtle issues that are worth bringing attention to. Consider the following figure:

<pre>if True: x = 3 print(x)</pre>	<pre>if (true) { int x = 3; } System.out.println(x);</pre>
--	--

Figure 5.9: Consequence of Statement Elaboration

Because variables in Java are local to the scope in which they were defined, x is not defined in the output statement and a type error is thrown. This issue is resolvable, but serves as a good example for the types of issues that can arise from poorly thought out elaboration.

5.3.2.1 Resolving Type Mutations

Resolving the issue of variable type mutability that arose in 5.2.3 is anything but trivial. It would certainly be reasonable to disallow the behavior altogether, but that would be a last resort, as we strive to stay as true to the source language as possible. Thus, Omni does a considerable amount of work in elaborating assignments to allow for variable type mutability in Python. Consider the following:

<pre>1 x = 12 2 x = True 3 x = "Squirrel" 4 x = 4</pre>	<pre>int x = 12; boolean A = True; String B = "Squirrel"; x = 4;</pre>
---	--

Figure 5.10: Type Mutations

When elaborating assignments, if the variable has already been declared, Omni will check if the type of the new assignment matches the type of the old assignment. If so, we do nothing, as in lines 1 and 4 of Figure 5.8. If not, we generate a new variable and add

the declaration.⁹ However, we are not completely finished. After generating a variable, we still need to update every expression after the assignment that used the old variable. After doing so, the process is complete.

5.3.2.2 Issues From Allowing Type Mutations

Unfortunately, allowing the expressiveness of variable type mutations comes at a price. Consider the following example:

<pre>x = True while x: x = 0 print(x)</pre>	<pre>boolean x = true; while (x) { int A = 0; System.out.println(A); }</pre>
---	--

Figure 5.11: Problems With Type Mutability

When both examples are run, the Python source code will stop and correctly output x once. However, the Java example will never exit the while loop. There are certainly more examples like this, but this was a simple, yet appropriately severe example. It is unclear how this issue would actually be resolved. However, rather than disallowing type mutations entirely, Omni could restrict type mutations to solely the general body of a function. By disallowing type mutations in, for example, loops and conditionals, we eliminate many of the issues at the cost of a slight reduction in expressiveness.¹⁰

⁹This is a very simplified version of the actual process.

¹⁰As these are very large languages, it is difficult to say with certainty that this proposed solution would resolve all the aforementioned issues, but, intuitively, it seems a reasonable resolution.

Chapter 6

Conclusions

Following elaboration, the final step in the process of Omni is to print out the transpilation. This is not a trivial task, but it is a fairly straightforward one and thus will not be discussed at length.

6.1 Features Excluded From Omni

It is no surprise that both Python and Java are very large languages. Due to time constraints, there are **many** features that are not supported by Omni. Notable ones include: data structures such as maps, sets, etc., classes and objects, packages and imports, and much more.

6.2 Potential Improvements to Omni

Omni strives to allow for as much expressiveness in code as it can. Thus, an important improvement would be to allow for method overloading as described in 5.2.4. Following that, improvements must be made to Omni's error messages before it can ever be considered an educational tool. Due to time constraints, most errors simply print out the type of error, potentially alongside any relevant information, such as the types in a type mismatch error or the expression in a bad conversion error. A very notable improvement would be to implement a unification-based type inference system, so that there is no need for explicit annotations whatsoever. However, as alluded to earlier, this would heavily complicate error messages, and thus would require a considerable amount of work to throw clear and helpful error messages.

6.3 Future Languages to Implement

Implementing any new languages is very far from being a consideration at the moment; however, C# would be a natural, and fairly simple, inclusion to Omni due to its similarities to Java. Another reasonable inclusion would be JavaScript. This would be more difficult to implement than C#, but introduces the issue of unsafe typing into Omni. Lastly, C++ would naturally be an excellent inclusion, but the prospect of managing memory allocation/deallocation is enough to dissuade attempting it for now.

Bibliography

- [1] Free Software Foundation. Gnu compiler collection. <https://gcc.gnu.org>. Accessed: 2023-11-18.
- [2] Niklas Broberg. `language-java`. <https://hackage.haskell.org/package/language-java>, 2018. Accessed: 2023-11-06.
- [3] Bernard James Pope. `language-python`. <https://hackage.haskell.org/package/language-python>, 2020. Accessed: 2023-11-06.
- [4] Brent A. Yorgey. Lecture on type systems, 2023.
- [5] Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, 2002.
- [6] Simon Peyton Jones. Type inference as constraint solving. <https://www.youtube.com/watch?v=-TJGhGa04F8>. Accessed: 2023-12-01.